

ITSAR2 313 – SYSTEM INTEGRATION AND ARCHITECTURE 2

LABORATORY REPORT: EXERCISE 1 — MONOLITH VS MICROSERVICE

MEMBERS: Andura, Carla C.

Bermejo, Kate Nicole

Garcia, Sophia Christi

Labrador, Mariene E.

Roma, Sean Justin E.

Abstract

This laboratory exercise involved building a Node.js monolithic application (Express-based) and a microservices system consisting of student-service, course-service, and enrollment-service using a structured architectural approach with repository interfaces. A central dashboard routes requests to either local in-memory data (monolithic mode) or HTTP-based services (microservices mode). The key finding is that the monolithic architecture is simpler at this scale—one codebase, one process, one deployment—while the microservices architecture demonstrates independent scalability, modularity, and fault isolation. The use of repository abstraction enabled a smooth transition between architectures without major code changes. The preferred architecture depends on context: monolithic for simplicity and rapid development, and microservices for scalability and distributed team environments.

I. INTRODUCTION

Monolithic architecture is a software design where all parts of an application—such as the user interface, business logic, and data access—are combined into a single, unified system. It uses one codebase and one database, with components closely connected through direct method calls and shared memory. Deployment is simple since the entire system can be launched with a single command. However, it lacks flexibility in scaling, as individual components cannot be scaled separately. Additionally, a failure in one part of the system can impact the entire application.

In contrast, microservices architecture breaks the application into smaller, independent services. Each service manages its own database and communicates with others through network protocols like HTTP. These services can be developed, tested, deployed, and scaled independently, improving fault isolation so that issues in one service do not necessarily affect others. However, this approach introduces added complexity, including the need for service discovery, managing communication between services, ensuring data consistency across systems, and implementing effective monitoring and observability.

In the traditional Model–View–Controller (MVC) pattern, the Controller interacts directly with the Model, often including database calls within its methods. The MVCR (Model–View–Controller–Repository) pattern enhances this by inserting a Repository Interface between the Controller and the Model. The Controller depends on this interface, while the framework’s service container provides the actual implementation. This setup allows the Controller to remain unaware of where the data originates—whether from a local database like SQLite or an external API. As a result, switching the backend source

(such as through an APP_BACKEND environment setting) only requires changing the service binding, without modifying the Controller's code.

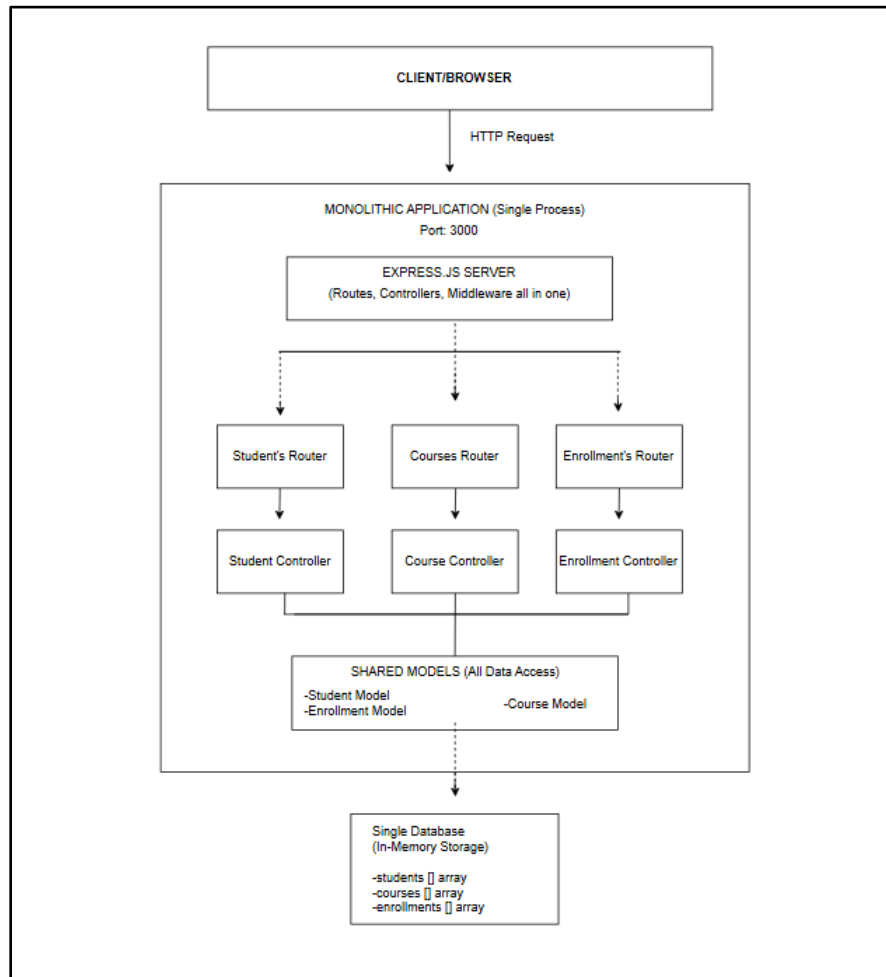
II. METHODOLOGY/IMPLEMENTATION

This section presents the system architecture of the Student Course System, illustrating both the monolithic and microservices implementations. The following diagrams provide a visual representation of how each architecture is structured, highlighting the flow of data, component interactions, and system boundaries.

The monolithic implementation is a single Node.js application running on an Express.js server (Port 3000). It follows a layered structure where routes, controllers, and middleware are handled within one unified process. The application contains separate routers for students, courses, and enrollments, each connected to their respective controllers. These controllers interact with shared models responsible for all data operations. All data is stored in a single in-memory database, where students, courses, and enrollments are maintained as arrays. Because everything runs in one codebase and process, communication between components happens through direct function calls, making the system simple to develop and deploy. However, all components are tightly coupled, meaning any failure or change can affect the entire system.

The microservices implementation separates the system into independent services: student-service (Port 4001), course-service (Port 4002), and enrollment-service (Port 4003). Each service runs as its own application and manages its own database (students.db, courses.db, enrollments.db). A central dashboard service acts as the frontend server, receiving client requests and forwarding them to the appropriate service via HTTP. The enrollment service performs internal validation by communicating with the student and course services to verify related data. Since services communicate over the network, failures such as connection issues or timeouts must be handled explicitly. This architecture improves scalability and fault isolation, as each service can be deployed and scaled independently. However, it introduces additional complexity in managing inter-service communication, data consistency, and system monitoring.

Figure 1: Monolithic Application Architecture Diagram



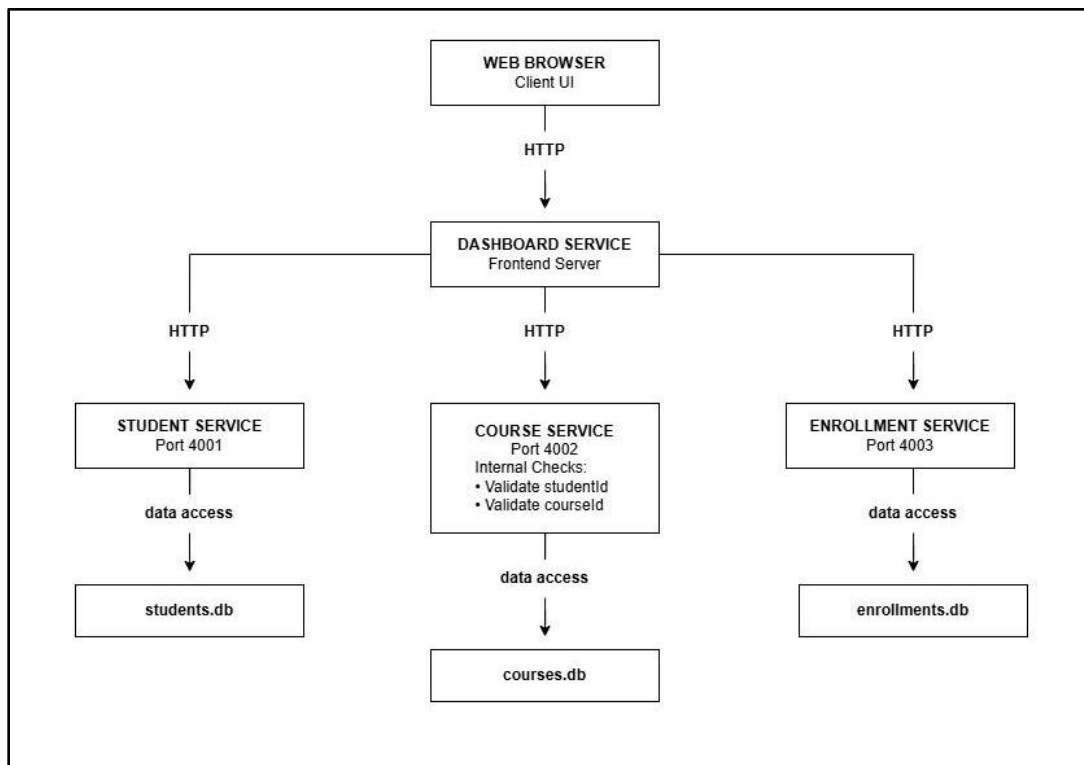


Figure 2: Microservices Application Architecture Diagram

Figure 3: Control Panel for Students

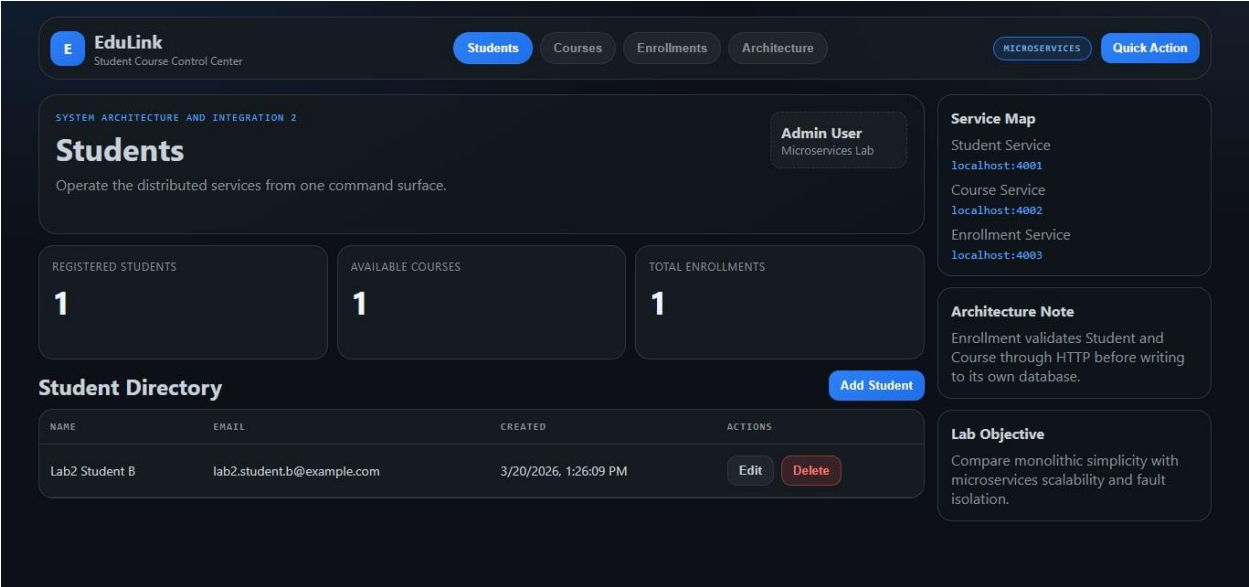


Figure 4: Control Panel for Courses

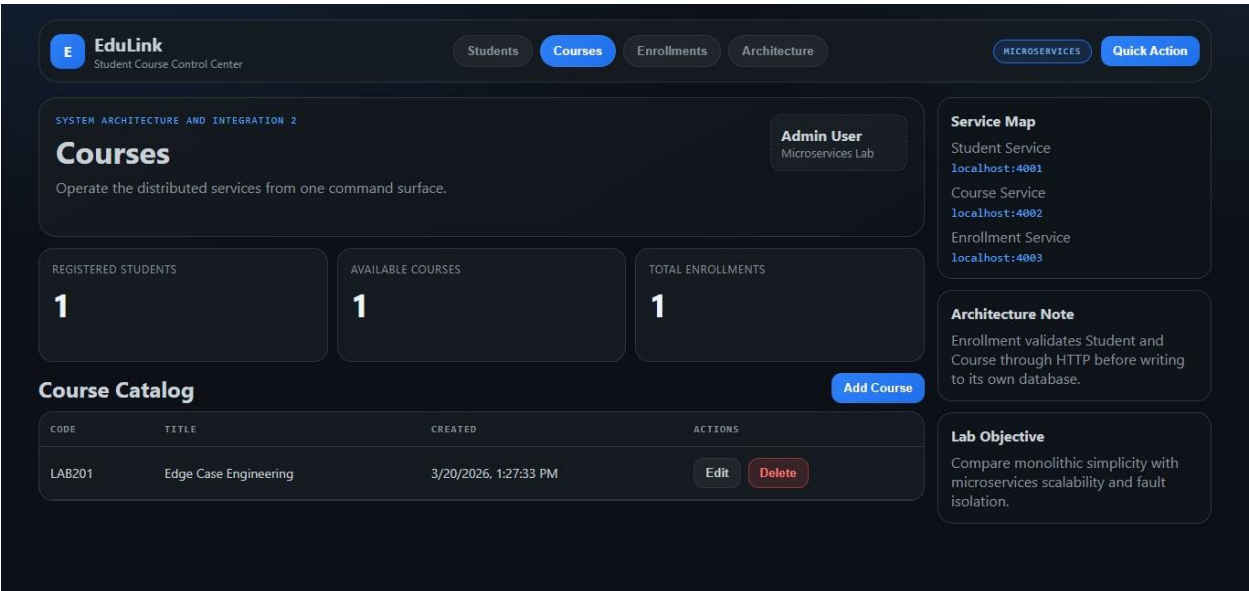
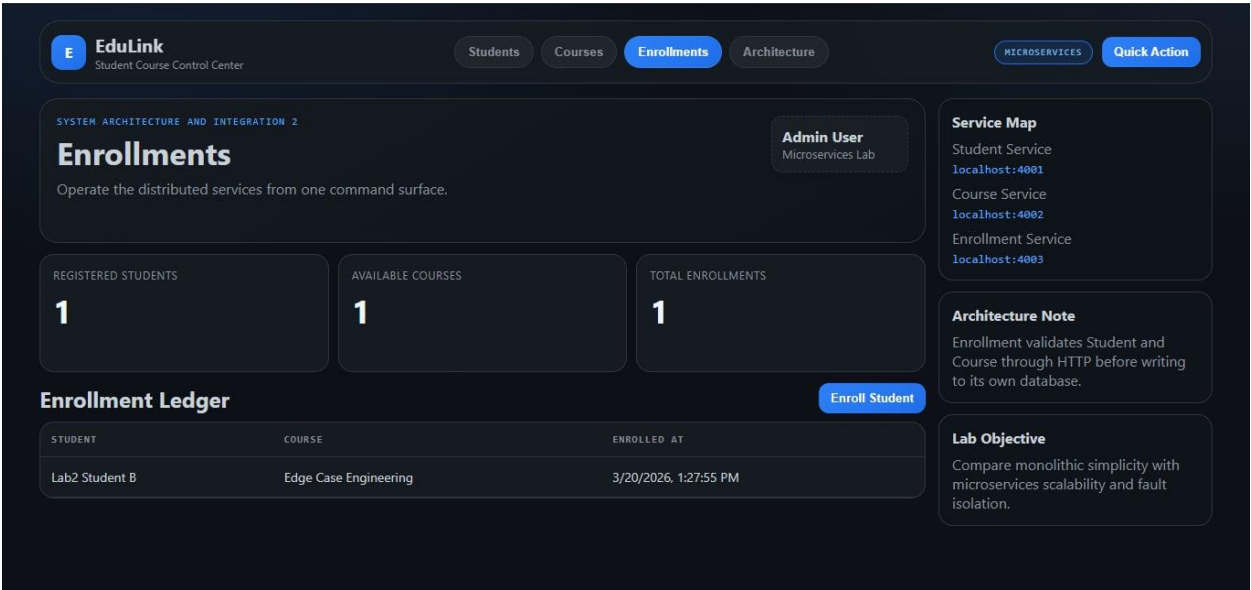


Figure 5: Control Panel for Enrollments



III. FINDINGS

Figure 6: Comparison Table

This table compares Monolithic and Microservices architectures across key aspects of system design.

Category	Monolithic Architecture	Microservices Architecture
Development	Fast & Simple: One codebase, one language, and unified debugging.	Complex: Requires managing multiple repos, languages, and APIs.
Deployment	Single Unit: One "push" deploys the entire application at once.	Independent: Each service has its own CI/CD pipeline and release cycle.

Scalability	Vertical/Bulk: You must scale the <i>entire</i> app, even if only one feature is busy.	Granular: Scale only the high-demand services (e.g., <i>Enrollment</i>).
Resilience	High Risk: A single memory leak or bug can crash the entire system.	Fault-Tolerant: A failure in one service (e.g., <i>Payments</i>) won't kill the others.
Performance	Low Latency: Fast, in-memory calls between internal modules.	Network Overhead: Slower due to HTTP/RPC calls between services.

In terms of development, monolithic systems are faster and simpler to build because everything exists in a single codebase, making debugging easier. In contrast, microservices are more complex since they require managing multiple services, repositories, and communication between APIs.

For deployment, a monolithic application is deployed as one complete unit, meaning any update requires redeploying the entire system. Microservices, however, allow each service to be deployed independently, enabling faster updates and continuous delivery.

Regarding scalability, monolithic systems scale as a whole, which can be inefficient if only one part is under heavy load. Microservices allow selective scaling, where only the needed services (e.g., enrollment or payments) are scaled.

In terms of resilience, monolithic systems are more vulnerable because a failure in one component can affect the entire application. Microservices improve fault tolerance since each service operates independently, so one failure does not impact others.

Lastly, for performance, monolithic systems typically have lower latency due to direct internal communication, while microservices may experience slight delays due to network communication between services using protocols like HTTP or RPC.

IV: REFLECTION

After implementing both the monolithic and microservices versions of the Student Course System, our team gained a clearer understanding that neither architecture is inherently superior. Instead, each approach offers different strengths depending on the project's scale, goals, and stage of

development. This project became less about choosing the “better” architecture and more about understanding when and why each one is appropriate.

Working with the monolithic implementation, which runs as a single Node.js application on an Express.js server, we appreciated how straightforward and efficient development can be. With everything in one codebase—routes, controllers, middleware, and models—development felt more direct and manageable. The use of shared in-memory data made it easier to implement features like students, courses, and enrollments without worrying about network communication. Everything communicated through direct function calls, which simplified both development and deployment.

At the same time, this simplicity also revealed its limitations. As the system grew, we realized how tightly coupled everything was. A change in one part of the system had the potential to affect the entire application. While cascade operations were easy to implement due to the shared database, this also highlighted how dependent the system was on a single structure. This experience helped us understand why monolithic systems are often best suited for smaller applications or early-stage development, where speed and simplicity are more important than scalability.

Another important realization came from the use of the Repository Interface, which played a key role in keeping our system flexible. By relying on interfaces instead of concrete implementations, we were able to separate our business logic from our data source. This made our controller methods clean and easy to maintain, and it also allowed us to transition between different implementations without rewriting the entire system. Looking back, this abstraction was essential in making our architecture more adaptable and future-ready.

Transitioning to the microservices architecture was a more challenging but insightful experience. By separating the system into independent services—student-service, course-service, and enrollment-service—we were introduced to the realities of distributed systems. Each service had its own database and ran independently, which required us to think differently about how components communicate. Instead of simple function calls, we relied on HTTP requests, which introduced concerns such as latency, network failures, and service availability.

One of the biggest lessons we learned was how data consistency becomes more complex in a distributed system. For example, operations like deleting a student required careful coordination between services. Unlike the monolith, where everything happens in a single transaction, the microservices approach requires us to think in terms of events and communication between services. This made us realize how much planning is needed to ensure data remains consistent across different systems.

We also observed how microservices improved our workflow as a team. Since each service was independent, we could divide responsibilities more effectively and work in parallel without constantly interfering with each other’s work. This modularity also improved fault isolation—if one service encountered an issue, it did not necessarily bring down the entire system. This was a powerful realization, especially in understanding how large-scale systems are built and maintained in real-world applications.

However, this flexibility came with added complexity. Managing multiple services, handling inter-service communication, and ensuring proper error handling required more effort compared to the

monolithic approach. We also had to deal with challenges like distributed logging, service monitoring, and maintaining consistency across multiple databases. This experience showed us that while microservices offer scalability and resilience, they also introduce a significant level of operational overhead.

Reflecting on the entire project, we realized that architecture decisions should always be guided by context. The monolithic approach was ideal for rapid development and simpler systems, while microservices became valuable when scalability, independence, and fault tolerance were required. This project not only strengthened our technical skills but also deepened our understanding of system design as a balance between simplicity and complexity.

In the end, this experience taught us that good architecture is not about choosing between monolithic or microservices—it is about understanding trade-offs and making informed decisions based on the needs of the system.

V. CONCLUSION

Our experience confirms that architectural maturity involves recognizing when to evolve from a centralized system to a distributed one. We believe the most effective strategy for modern software development is often a "Monolith-First" approach, followed by a strategic decomposition into microservices as organizational and technical demands grow. This hands-on comparison has provided our team with a comprehensive understanding of the trade-offs that define professional software engineering, moving us beyond code implementation and into the realm of system architecture.